IFCD107 · ESPECIALISTA EN INTELIGENCIA ARTIFICIAL

ETAPA B · CONSTRUIR · SPRINT 8 DE 17

# Sprint 8 · Árboles, ensembles y SVM

*¿cuál es el mejor modelo y cuánto nos cuesta mantenerlo?*

**FECHAS**
9–11 de noviembre

**HORAS**
11 h (3 h teoría · 5 h práctica guiada · 3 h PBL)

**CONTENIDOS**
M3 UD 3.6 (árboles de decisión), UD 3.7 (Random Forest, Gradient Boosting, XGBoost y LightGBM) y UD 3.8 (SVM), más ajuste de hiperparámetros (`GridSearchCV`, `RandomizedSearchCV`).

**EVALUACIÓN**
test de M3 (primera parte) y reto PBL «Duelo de modelos».

## Qué aprenderás

1. Entrenar un **árbol de decisión**, detectar su sobreajuste, podarlo y leer sus reglas y su importancia de variables.
2. Explicar la diferencia entre **bosque aleatorio** (promediar árboles) y **boosting** (corregir errores en cadena), y usar XGBoost y LightGBM con parada temprana.
3. Ajustar una **SVM** (`C`, `gamma`, kernel) y saber cuándo su coste la descarta.
4. Buscar hiperparámetros con **`GridSearchCV`** y **`RandomizedSearchCV`** sin engañarte con la nota de validación.
5. **Elegir un modelo campeón** justificando rendimiento, explicabilidad y coste, y documentarlo en una *model card*.

## Requisitos previos

- Sprint 6 y 7: pipelines, validación cruzada, métricas de clasificación (AUC, precisión, recall), umbral y regresión logística como referencia.
- Datos: `datos/reservas_turisdata.csv` (objetivo `cancelada`). Los notebooks lo descargan solos si no lo encuentran.
- Librerías: scikit-learn, xgboost y lightgbm (vienen instaladas en Colab; en local: `pip install xgboost lightgbm`).

## Prácticas

| Notebook | Tema | Tiempo | Colab |
|---|---|---|---|
| `S08_01_arboles_decision` | Árbol de decisión: reglas, sobreajuste, poda, importancia | 55 min | Abrir en Colab |
| `S08_02_random_forest` | Random Forest: bagging, OOB, importancia por permutación | 55 min | Abrir en Colab |
| `S08_03_boosting_xgboost_lightgbm` | Gradient Boosting, XGBoost, LightGBM y parada temprana | 65 min | Abrir en Colab |
| `S08_04_svm` | SVM: margen, kernels, `C`, `gamma`, escalado y coste | 55 min | Abrir en Colab |
| `S08_05_busqueda_hiperparametros` | `GridSearchCV`, `RandomizedSearchCV`, `cv_results_`, test final | 60 min | Abrir en Colab |
| `S08_06_leaderboard_plantilla` | **Herramienta del reto**: leaderboard para rellenar | 45 min | Abrir en Colab |

Las cinco primeras son las prácticas guiadas; la sexta es la plantilla que usaréis en el reto. Los ejercicios se autocorrigen con `assert`.

## RETO Reto PBL · «Duelo de modelos» (3 h, en equipo)

**Contexto del cliente.** TurisData ya sabe que puede predecir cancelaciones (Sprint 7). Ahora quiere saber **qué modelo poner en producción**. El gerente pregunta: «¿el más preciso? ¿el que puedo entender? ¿el más barato de mantener?». Vuestro equipo compite consigo mismo: tenéis que enfrentar **árboles, bosques, boosting y SVM**, y **elegir un campeón** con argumentos.

**Qué se entrega.**

1. **Leaderboard**: `leaderboard_S08.csv` con al menos **6 modelos** (baseline y logística incluidos) medidos igual. Se rellena con el notebook `S08_06_leaderboard_plantilla` (plantilla del CSV: `guias/S08/leaderboard_plantilla.csv`).
2. **Model card** del campeón: `model_card.md` siguiendo `guias/S08/model_card_plantilla.md` (1–2 páginas).
3. `reto_S08.ipynb` con la comparación reproducible y la búsqueda de hiperparámetros de, al menos, **dos** familias.
4. `README.md` para reproducirlo (versiones, semilla, orden de ejecución).

**Restricciones.**

- Mismos datos, mismos folds y misma métrica para todos los modelos (ya lo garantiza la plantilla).
- La búsqueda de hiperparámetros se hace **solo sobre train**. **El test se usa una vez**, únicamente con el campeón.
- Máximo 8 modelos en el leaderboard; tiempo máximo de cada búsqueda: 5 minutos en Colab.
- Solo librerías del curso. Semilla fija ( `SEMILLA = 42` ).
- El campeón se justifica con **tres** criterios: **rendimiento** (AUC con incertidumbre), **explicabilidad** (¿puede el gerente entender por qué?) y **coste** (tiempo, tamaño, mantenimiento).

**Criterios de evaluación** (0–4 cada uno; misma rúbrica de todos los retos):

| Criterio | 1 · Insuficiente | 2 · Suficiente | 3 · Bueno | 4 · Excelente |
|---|---|---|---|---|
| Corrección técnica | Errores que invalidan el resultado | Correcto con fallos menores | Correcto y bien justificado | Correcto, justificado y con alternativas comparadas |
| Reproducibilidad | No se puede volver a ejecutar | Se ejecuta con ayuda | Se ejecuta desde el README | Además incluye datos o script y versiones fijas |
| Análisis y comunicación al cliente | Solo código o cifras sin interpretar | Interpretación básica | Conclusiones claras para el cliente | Recomendación accionable con límites explicados |
| IA responsable | No se considera | Menciones genéricas | Riesgos específicos identificados | Medidas aplicadas y evaluadas |

*Pistas de IA responsable (en la model card):* usos previstos y **no** previstos, qué variables sensibles hay (país de origen), qué pasa si el comportamiento de los clientes cambia (deriva), cómo se vigilará el modelo y quién revisa las decisiones.

**Pasos sugeridos.**

- **Día 1 (9/11):** *planning* (30 min); acordar los **pesos** de rendimiento/explicabilidad/coste con la «voz del cliente»; registrar baseline, logística, árbol podado y bosque en el leaderboard.
- **Día 2 (10/11):** boosting y SVM; búsqueda de hiperparámetros (rejilla o aleatoria) para las dos familias más prometedoras; actualizar el leaderboard con los mejores modelos.
- **Día 3 (11/11):** análisis de sensibilidad de los pesos, elección del campeón, evaluación en test **una vez**, *model card*, README, autoevaluación y coevaluación; **review + retrospectiva** (última hora).

**Qué se enseña en la review** (5 min por equipo): el leaderboard (tabla o gráfico), el campeón y **por qué no ganó otro**, y una frase sobre qué cambiaría la elección si el cliente valorara otra cosa.

**Preguntas de la retrospectiva:** ¿nos sorprendió que un modelo sencillo compitiera con uno complejo? ¿Qué criterio pesó más en nuestra decisión? ¿Qué mediríamos en producción para saber que el modelo se degrada?

## Definition of Done

- [ ] `leaderboard_S08.csv` con ≥ 6 modelos de al menos 4 familias (árbol, ensemble, SVM y lineal/baseline).
- [ ] Búsqueda de hiperparámetros con validación cruzada sobre train para al menos dos familias.
- [ ] Campeón elegido con los tres criterios y análisis de sensibilidad de los pesos.
- [ ] AUC de test calculado **una sola vez**, solo para el campeón.
- [ ] `model_card.md` completa (usos, datos, métricas, limitaciones, IA responsable, mantenimiento).
- [ ] Notebook ejecutable de principio a fin y README con versiones.
- [ ] Subido a Git; autoevaluación y coevaluación rellenadas.

## Recursos

- Documentación de scikit-learn: *Ensemble methods*, *Decision Trees* y *Support Vector Machines*.
- Documentación de XGBoost y de LightGBM: *Parameters Tuning* (guía de ajuste de hiperparámetros).
- «Model Cards for Model Reporting» (Mitchell et al., 2019): artículo que da origen a las model cards.
- Guía de scikit-learn: *Tuning the hyper-parameters of an estimator*.
- Tu propio notebook `S07_03` (umbral y coste) y el informe del Sprint 7: la recomendación de negocio sigue en pie.